

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 1

Warmup

Exercise

- ▶ How fast can we query/insert with these data structures?

	Query	Insert
Unsorted linked list		
Unsorted array		
Sorted array		
BST		

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 5 | Part 2

Direct Address Tables

Counting Frequencies

- How many times does each age appear?

PID	Name	Age
A1843	Wan	24
A8293	Deveron	22
A9821	Vinod	41
A8172	Aleix	17
A2882	Kayden	4
A1829	Raghu	51
A9772	Cui	48
⋮	⋮	⋮

Exercise

What data structure would you use to store the age counts?

Direct Address Tables

- ▶ Idea: keep an **array** `arr` of length, say, 125.
- ▶ Initialize to zero.
- ▶ If we see age `x`, increment `arr[x]` by one.

Building the Table

```
# loading the table  
table = np.zeros(125)
```

```
for age in ages:  
    table[age] += 1
```

- ▶ Time complexity if there are n people?

Query

```
# query: how many people are 55?  
print(table[55])
```

- Time complexity if there are n people?

Counting Names

- How many times does each name appear?

PID	Name	Age
A1843	Wan	24
A8293	Deveron	22
A9821	Vinod	41
A8172	Aleix	17
A2882	Kayden	4
A1829	Raghu	51
A9772	Cui	48
⋮	⋮	⋮

Downsides

- ▶ DATs are **fast**.
- ▶ What are the downsides of DATs?
- ▶ Could we use a DAT to store:
 - ▶ zip codes?
 - ▶ phone numbers?
 - ▶ credit card numbers?
 - ▶ names?

Downsides

- ▶ Things being stored must be integers, or convertible to integers
 - ▶ why? valid array indices
- ▶ Must come from a small range of possibilities
 - ▶ why? memory usage. example: phone numbers

Hash Tables

- ▶ Insight: anything can be “converted” to an integer through **hashing**.
- ▶ But not uniquely!
- ▶ Hash tables have many of the same advantages as DATs, but work more generally.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 3

Hashing

Hashing

- ▶ One of the most important ideas in CS.
- ▶ Tons of uses:
 - ▶ Verifying message integrity.
 - ▶ Fast queries on a large data set.
 - ▶ Identify if file has changed in version control.

Hash Function

- ▶ A **hash function** takes a (large) object and returns a (smaller) “fingerprint” of that object.
- ▶ Usually the fingerprint is a number, guaranteed to be in some range.

How?

- ▶ Looking at certain bits, combining them in ways that *look* random (but aren't!)

Hash Function Properties

- ▶ Hashing same thing twice returns the same hash.
- ▶ Unlikely that different things have same fingerprint.
 - ▶ But not impossible!

Collisions

- ▶ Hash functions map objects to numbers in a defined range.
 - ▶ Example: given image, return number in $[0, 1, 2, \dots, 1024]$
- ▶ There will be two images with the same hash.
 - ▶ **Pigeonhole principle**: if there are n pigeons, $< n$ holes, there will a hole with ≥ 2 pigeons.
- ▶ **Collision**: two objects have the same hash

“Good” Hash Functions

- ▶ A good hash function tries to minimize collisions.

Hashing in Python

- The `hash` function computes a hash.

```
>>> hash("This is a test")  
-670458579957477203  
>>> hash("This is a test")  
-670458579957477203  
>>> hash("This is a test!")  
1860306055874153109
```

MD5

- ▶ MD5 is a **cryptographic** hash function.
 - ▶ Hard to “reverse engineer” input from hash.
- ▶ Returns a *really large* number in hex.

a741d8524a853cf83ca21eabf8cea190

- ▶ Used to “fingerprint” whole files.

Example

```
> echo "My name is Akbar" | md5
a741d8524a853cf83ca21eabf8cea190
> echo "My name is Akbar" | md5
a741d8524a853cf83ca21eabf8cea190
> echo "My name is Akbar!" | md5
f11eed2391bbd0a5a2355397c089fafd
```

Example

```
> md5 slides.pdf  
e3fd4370fda30ceb978390004e07b9df
```

Why?

- ▶ I release a piece of software.
- ▶ I host it on Google Drive.
- ▶ Someone (Google, US Gov., etc.) decides to insert extra code into software to spy on users.
- ▶ You have no way of knowing.

Why?

- ▶ I release a piece of software & **publish the hash**.
- ▶ I host it on Google Drive.
- ▶ Someone inserts extra code.
- ▶ You download the software and hash it. If hash is different, you know the file has been changed!

Hashing for us

- ▶ Don't need to know much about *how* the hash function works.
- ▶ But should know how they are used.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 5 | Part 4

Hash Tables

Membership Queries

- ▶ **Given:** a collection of n numbers and a target t .
- ▶ **Find:** determine if t is in the collection.

Goal

- ▶ DATs are fast, but won't work for things that aren't numbers in a small range.
- ▶ Idea: hash objects to numbers in a small range, use a DAT.
- ▶ But must deal with collisions.

Hash Tables

- ▶ Pick a table size m .
 - ▶ Usually $m \approx$ number of things you'll be storing.
- ▶ Create hash function to turn input into a number in $\{0, 1, \dots, m - 1\}$.
- ▶ Create DAT with m bins.

Example

hash('hello') == 3

hash('data') == 0

hash('science') == 4

0 1 2 3 4 ... $m - 1$

Collisions

- ▶ The **universe** is the set of all possible inputs.
- ▶ This is usually much larger than m (even infinite).
- ▶ Not possible to assign each input to a unique bin.
- ▶ If `hash(a) == hash(b)`, there is a **collision**.

Example

hash('hello') == 3

hash('data') == 0

hash('san diego') == 3

0 1 2 3 4 ... $m - 1$

Chaining

- ▶ Collisions stored in same bin, in linked list.
- ▶ **Query:** Hash to find bin, then linear search.



The Idea

- ▶ A good hash function will utilize all bins evenly.
 - ▶ Looks like uniform random distribution.
- ▶ If $m \approx n$, then only a few elements in each bin.
- ▶ As we add more elements, we need to add bins.

Average Case

- ▶ n elements in table.
- ▶ m bins.
- ▶ Assume elements placed randomly in bins¹.
- ▶ Expected bin size: n/m

¹Of course, they are placed deterministically.

Analysis

- ▶ Query:
 - ▶ Time to find correct bin: $\Theta(1)$
 - ▶ Expected number of elements in the bin: n/m
 - ▶ Time to perform linear search: $\Theta(n/m)$
 - ▶ Total: $\Theta(1 + n/m)$
 - ▶ We usually guarantee $m = \Theta(n)$
 - ▶ Expected time: $\Theta(1)$.

Worst Case

- ▶ Everything hashed to same bin.
 - ▶ Really unlikely!
- ▶ Query:
 - ▶ $\Theta(1)$ to find bin
 - ▶ $\Theta(n)$ for linear search.
 - ▶ Total: $\Theta(n)$.

Exercise

What is the worst case time complexity of inserting an element into a hash table that uses chaining with linked lists?

Growing the Hash Table

- ▶ Insertions take $\Theta(1)$ **unless** the hash table needs to grow.
- ▶ We need to ensure that $m \leq c \cdot n$.
 - ▶ Otherwise, too many collisions.
- ▶ If we add a bunch of elements, we'll need to increase m .
- ▶ Increasing m means allocating a new array, $\Theta(m) = \Theta(n)$ time.

Main Idea

Hash tables support constant (expected) time insertion and membership queries.

Dictionaryes

- ▶ Hash tables can also be used to store (key, value) pairs.
- ▶ Often called **dictionaries** or **associative arrays**.

Hashing in Python

- ▶ `dict` and `set` implement hash tables.
- ▶ Querying is done using `in`:

```
>>> # make a set
>>> L = {3, 6, -2, 1, 7, 12}
>>> 4 in L # Theta(1)
False
>>> 7 in L # Theta(1)
True
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 5

Fast Algorithms with Hash Tables

Faster Algorithms

- ▶ Hashing is a super common trick.
- ▶ The “best” solution to interview problems often involves hashing.

Example 1: The Movie Problem

- ▶ You're on a flight that will last D minutes.
- ▶ You want to pick two movies to watch.
- ▶ Find two whose durations sum to **exactly** D .

Recall: Previous Solutions

- ▶ Brute force: $\Theta(n^2)$.
- ▶ Sort, use sorted structure: $\Theta(n \log n) + \Theta(n)$.
- ▶ Theoretical lower bound: $\Omega(n)$?
- ▶ Can we speed this up with hash tables?

Idea

- ▶ To use hash tables, we want to frame problem as a **membership query**.

Example

- ▶ Suppose flight is 360 minutes long.
- ▶ Suppose first movie is fixed: 120 minutes.
- ▶ Is there a movie lasting $(360 - 120) = 140$ minutes?

```
def optimize_entertainment_hash(times, D):  
    hash_table = dict()  
    for i, time in enumerate(times):  
        hash_table[time] = i  
  
    for i, time in enumerate(times):  
        target = D - time  
        if target in hash_table:  
            return i, hash_table[target]
```

Example 2: Anagrams

Definition

Two strings w_1 and w_2 are **anagrams** if the letters of w_1 one can be permuted to make w_2 .

Examples

- ▶ abcd / dbca
- ▶ listen / silent
- ▶ sandiego / doginsea

Problem

- ▶ Given a collection of n strings, determine if any two of them are anagrams.

Exercise

Design an efficient algorithm for solving this problem.
What is its time complexity?

Solution

- ▶ We need to turn this into a **membership query**.
- ▶ **Trick:** two strings are anagrams iff

`sorted(w_1) == sorted(w_2)`

```
def any_anagrams(words):  
    seen = set()  
    for word in words:  
        w = sorted(word)  
        if w in seen:  
            return True  
        else:  
            seen.add(w)
```


Hashing **Downsides**

- ▶ Problem must involve **membership query**.

Example: The Movie Problem

- ▶ You're on a flight that will last D minutes.
- ▶ You want to pick two movies to watch.
- ▶ Find two whose added durations is **closest** to D .

Hashing **Downsides**

- ▶ No locality: similar items map to different bins.
- ▶ There is no way to quickly query entry closest to given input.

Example: Number of Elements

- ▶ Given a collection of n numbers and two endpoints, a and b , determine how many of the numbers are contained in $[a, b]$.
- ▶ Not a membership query.
- ▶ Idea: **sort** and use modified binary search.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 6

Hash Table Drawbacks

Hashing **Downsides**

- ▶ No locality: similar items map to different bins.
- ▶ But we often want similar items at the same time.
- ▶ Results in many **cache misses**, **slow**.

Hashing **Downsides**

- ▶ Memory overhead.

Hash Tables vs. BSTs

- ▶ Hash Table: $\Theta(1)$ insertion, query (expected time).
- ▶ BST: $\Theta(\log n)$ insertion, query (if balanced).
- ▶ Why ever use a BST?

Hash Tables vs. BSTs

- ▶ Hash tables keep items in arbitrary order.
- ▶ Example: how many elements are in the interval $[3, 23]$?
- ▶ Example: what is the min/max/median?
- ▶ BSTs win when order is important.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 7

Approximate Nearest Neighbors (ANN)

ANN

- ▶ **Given:** A set of points and a query point, p .
- ▶ **Return:** An **approximate nearest neighbor**.

Today

- ▶ **Approximate nearest neighbors** via **Locality Sensitive Hashing (LSH)**.

CS-GY 6033

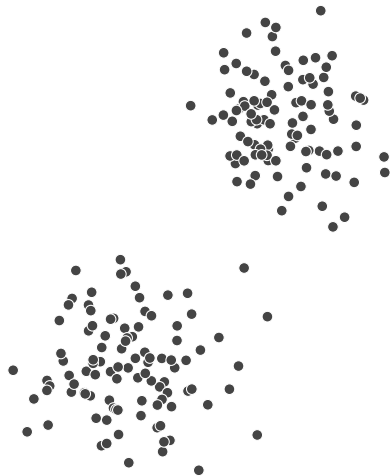
Design and Analysis of Algorithms I

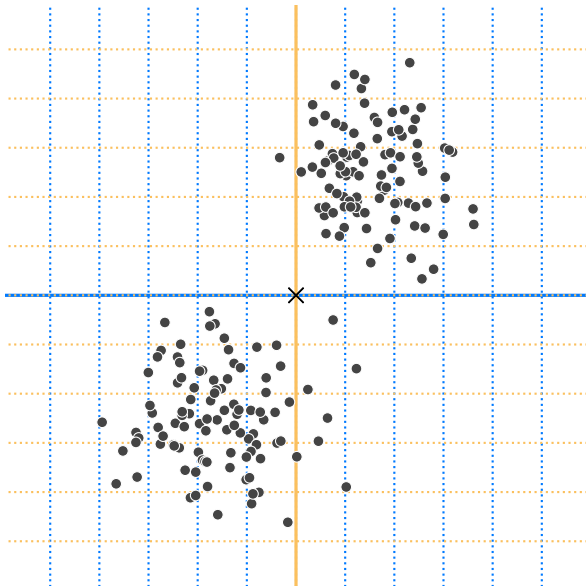
Lecture 5 | Part 8

Implementing a NN Grid

Grids

- ▶ Given input point p , want quick way to find nearby points.
- ▶ Idea: divide space into cells using grid.
- ▶ Find cell containing p , search it.
- ▶ How would we implement this?





Grid Cells

- ▶ Each point (x, y) given cell id: $(\lfloor x \rfloor, \lfloor y \rfloor)$
 - ▶ Example: $(1.2, 6.7)$ given cell id $(1, 6)$.
- ▶ Store (x, y) in dictionary with cell id as key.
 - ▶ Discretization allows multiple points in same cell.
 - ▶ Store collisions in list.
- ▶ Generalizes naturally to d -dimensions.

```
class NNGrid:
```

```
    def __init__(self, width):  
        self.width = width  
        self.cells = {}
```

```
    def cell_id(self, p):  
        p = np.asarray(p)  
        cell_id = np.floor(p / self.width).astype(int)  
        return tuple(cell_id)
```

```
    def insert(self, p):  
        """Insert p into the grid."""  
        cell_id = self.cell_id(p)  
        if cell_id not in self.cells:  
            self.cells[cell_id] = []  
        self.cells[cell_id].append(p)
```

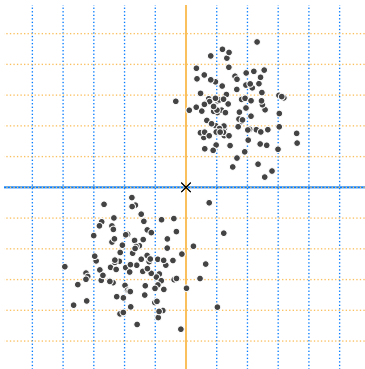
```
...
```

...

```
def points_in_cell(self, p):  
    cell_id = self.cell_id(p)  
    if cell_id not in self.cells:  
        return []  
    points_in_cell = self.cells[cell_id]  
    # turn into an array  
    return np.vstack(points_in_cell)  
  
def query(self, p):  
    return brute_force_nn(self.points_in_cell(p), p)
```

Note

- This may **fail** – NN could be in different cell.



Problems

- ▶ In d dimensions, cell id has d entries.

$$\text{cell-id}(p) = (\lfloor x_1/w \rfloor, \lfloor x_2/w \rfloor, \dots, \lfloor x_d/w \rfloor)$$

- ▶ All entries must be **exactly** the same for two points to have same cell id.
- ▶ This is **very unlikely**. Most cells are empty or contain one point.

High-Dimensional Cuboids

- ▶ One “fix”: increase cell width parameter.
- ▶ Suppose we want it to be likely that any points within distance r are in same cell.
- ▶ Then cell width should be $\approx 2r$.

High-Dimensional Cuboids

- ▶ But a d -dimensional cuboid of width $2r$ can contain points at distance $2\sqrt{d}r$ from one another!
- ▶ For even modest r , the whole data set is in one cell.

Main Idea

Dividing into a grid of cuboids fails in high dimensions. Either the cells are empty, or contain everything, depending on the width!

CS-GY 6033

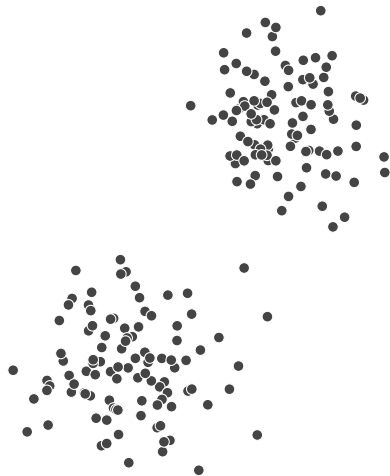
Design and Analysis of Algorithms I

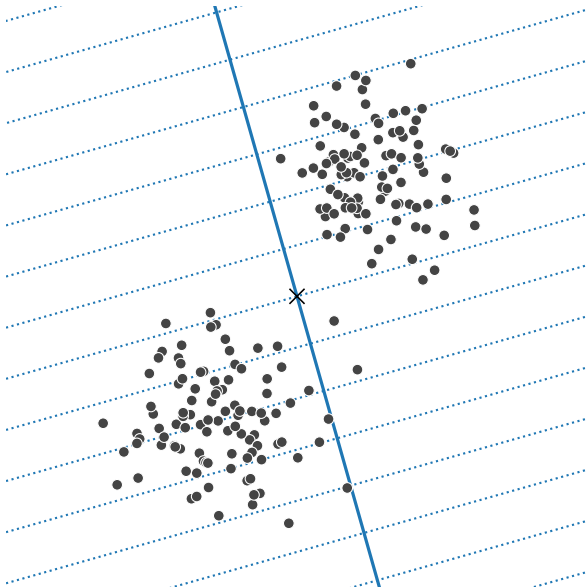
Lecture 5 | Part 9

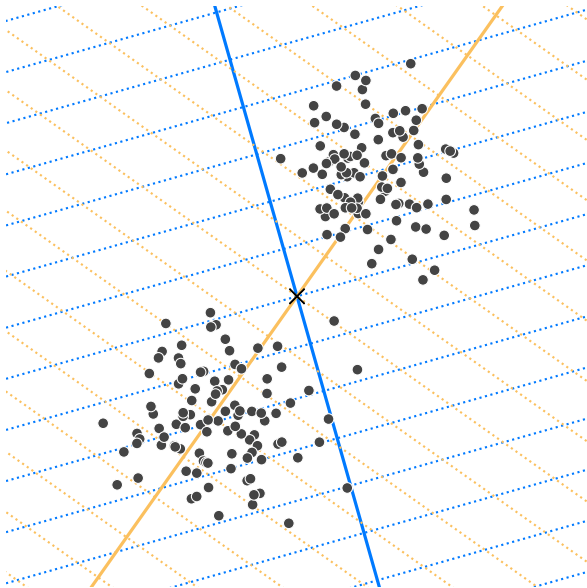
A Randomized “Grid”

A Randomized “Grid”

- Idea: Instead of axis-aligned grid, divide into cells using $k \ll d$ random directions.

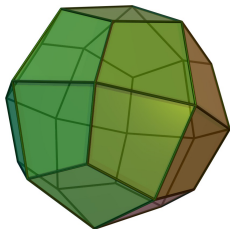






Cell Shape

- ▶ Cells are no longer d -dimensional cuboids.
- ▶ They are random k -dimensional **polytopes**.

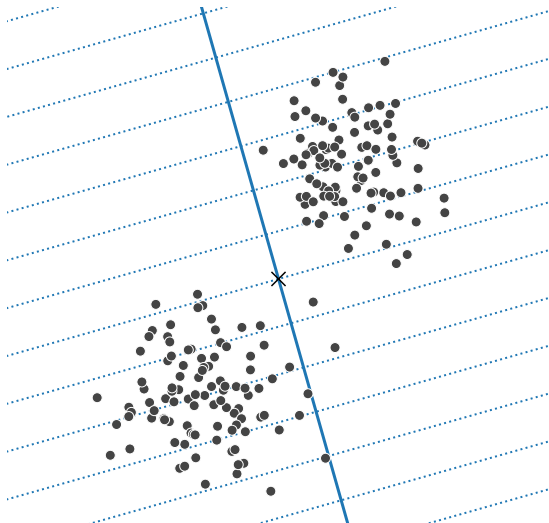


Question

- ▶ Why is this better? We'll see in the next sections.

Projection

- How do we determine which cell a point lies in?



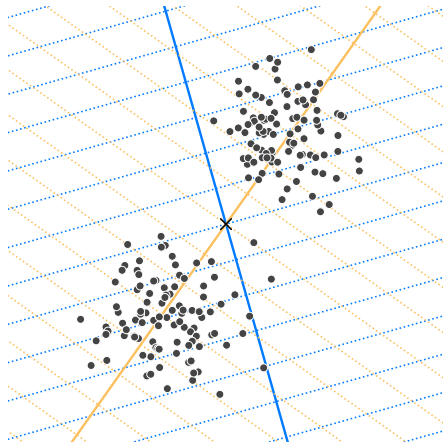
Cell IDs

- ▶ Pick k random unit vectors, $\vec{u}^{(1)}, \dots, \vec{u}^{(k)} \in \mathbb{R}^d$.
- ▶ Pick a width parameter, w .
- ▶ Given any point $\vec{p}$, its cell id is²:

$$\text{cell-id}(\vec{p}) = \left(\left\lfloor \frac{\vec{u}^{(1)} \cdot \vec{p}}{w} \right\rfloor, \left\lfloor \frac{\vec{u}^{(2)} \cdot \vec{p}}{w} \right\rfloor, \dots, \left\lfloor \frac{\vec{u}^{(k)} \cdot \vec{p}}{w} \right\rfloor, \right)$$

²use same width and unit vectors for all points

Example



Quick Cell-ID Calculation

- Place $\vec{u}^{(1)}, \dots, \vec{u}^{(k)}$ into a matrix:

$$U = \begin{pmatrix} \leftarrow & (\vec{u}^{(1)})^T & \rightarrow \\ \leftarrow & (\vec{u}^{(2)})^T & \rightarrow \\ \vdots & \vdots & \vdots \\ \leftarrow & (\vec{u}^{(k)})^T & \rightarrow \end{pmatrix}$$

- Then $\text{cell-id}(\vec{p}) = \text{entrywise-floor}(U\vec{p}/w)$

Generating Random Unit Vectors

```
def gaussian_projection_matrix(k, d):  
    X = np.random.normal(size=(k, d))  
    U = X / np.linalg.norm(X, axis=1)[: ,None]  
    return U
```

```
class NNProjectionGrid
```

```
    def __init__(self, projection_matrix, width):  
        self.width = width  
        self.projection_matrix = projection_matrix  
        self.cells = {}
```

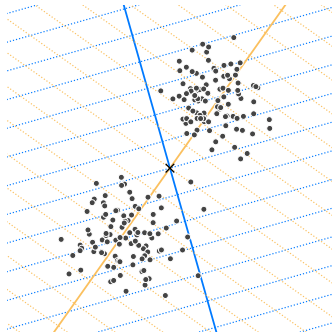
```
    def cell_id(self, p):  
        projection = self.projection_matrix @ p  
        cell_id = np.floor(projection / self.width)  
        return tuple(cell_id.astype(int))
```

```
# insert, query, points_in_cell same as for NNGrid
```

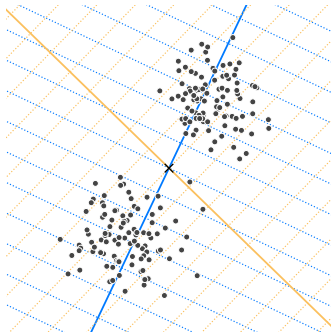
But wait...

- ▶ In high dimensions, still **very unlikely** for cell to contain >1 point.
- ▶ Idea: **banding**. Try, try again.
- ▶ Build multiple NNProjectionGrids with different random projections.
- ▶ Find `points_in_cell` for each, pool them together.

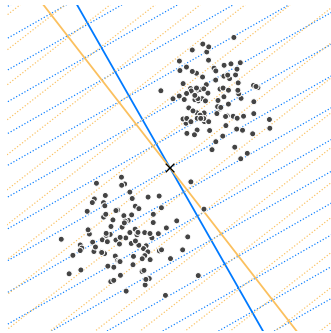
Multiple Random Projections



U_1



U_2



U_3

Locality Sensitive Hashing

- ▶ This idea (multiple random projections) is an example of **Locality Sensitive Hashing** (LSH).
- ▶ We'll explore it more in the next section.


```
class LocalitySensitiveHashing:

    def __init__(self, l, k, d, w):
        self.randomized_grids = []
        for i in range(l):
            U = gaussian_projection_matrix(k, d)
            randomized_grid = NNProjectionGrid(U, w)
            self.randomized_grids.append(randomized_grid)

    def insert(self, p):
        for randomized_grid in self.randomized_grids:
            randomized_grid.insert(p)

    ...
```

...

```
def query_close(self, p):
    nearby = []
    for randomized_grid in self.randomized_grids:
        points_in_cell = randomized_grid.points_in_cell(p)
        nearby.append(points_in_cell)
    return np.vstack(nearby)

def query_nn(self, point):
    results = self.query_close(point)
    pool = np.vstack([r for r in results])
    if len(pool) == 0:
        raise ValueError('No points nearby.')
    return brute_force_nn(pool, point)
```

Parameters

- ▶ l : number of randomized “grids”
- ▶ k : number of random directions in each “grid”
- ▶ w : bin width

Tuning Parameters

- ▶ Choose so that `.query_close` returns a small # of points.
- ▶ If # is very small (or zero), either:
 - ▶ increase w or ℓ
 - ▶ decrease k

Note

- ▶ This is an approximate NN technique!
- ▶ May not find **the** NN.
- ▶ May not return **anything**!

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 10

Theory of Locality Sensitive Hashing

Why does LSH work?

- ▶ Two approaches to understanding LSH.
- ▶ **1) Hashing view.**
- ▶ 2) Dimensionality reduction view.

Standard Hashing

- ▶ A **hash function** $h : \mathcal{X} \rightarrow \mathbb{Z}$ takes in an object from $\mathcal{X}$ and returns a bucket number.

Standard Hashing

- ▶ **Collision**: two different objects have same hash.
- ▶ Usually, collisions are **bad**.
- ▶ Want similar things to have very different hashes.

Locality Sensitive Hashing

- ▶ But in NN search, we want “close” items to be in the **same bucket** (have same hash).
- ▶ “Far” items should be in **different buckets** (have different hash).

Locality Sensitive Hashing

- ▶ Let r be a distance we consider “close”.
- ▶ Let cr (with $c > 1$) be a distance we consider “far”.
- ▶ Suppose H is a **family** of hash functions.

LSH Family

- ▶ H is an **LSH family** if when h is randomly drawn from H :

$$\begin{aligned}\mathbb{P}(h(x) = h(y)) &\geq p_1 && \text{when } d(x, y) \leq r \\ \mathbb{P}(h(x) = h(y)) &\leq p_2 && \text{when } d(x, y) \geq cr\end{aligned}$$

where $p_1 > p_2$.

Main Idea

If x and y are close, the probability that they hash to the **same** bin is not too small. If they are far, the probability is not too large.

Example: Random Projections

- ▶ We have seen one LSH family: random projections followed by binning.
- ▶ H has infinitely-many hash functions, one for each direction $\vec{u}$:

$$h_{\vec{u}}(\vec{p}) = \left\lfloor \frac{\vec{u} \cdot \vec{p}}{w} \right\rfloor,$$

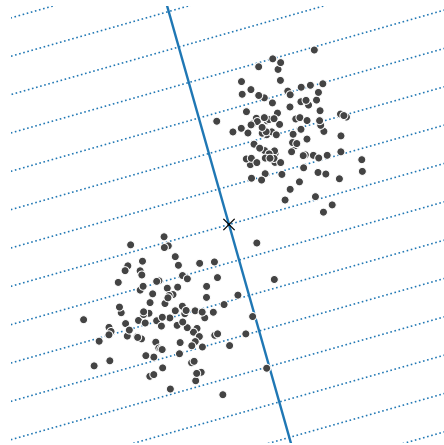
Example: Random Projections

- ▶ Suppose a random hash function h is chosen.
- ▶ Claim:

$$\mathbb{P}(h(x) = h(y)) \geq \frac{1}{2} \quad \text{when } d(x, y) \leq w/2$$

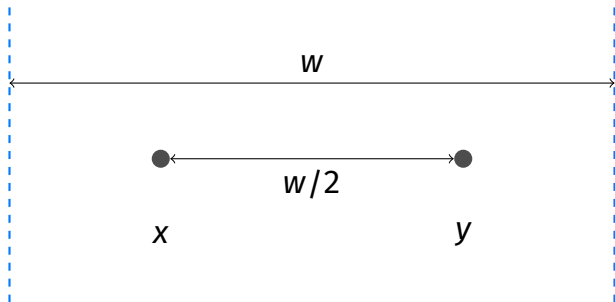
$$\mathbb{P}(h(x) = h(y)) \leq \frac{1}{3} \quad \text{when } d(x, y) \geq 2w$$

Intuition



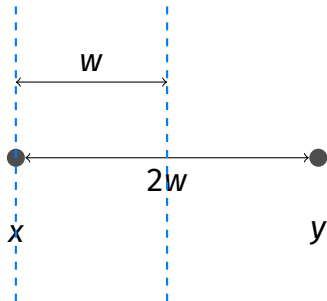
Proof: Close

- In worst case, grid is orthogonal to line between points.



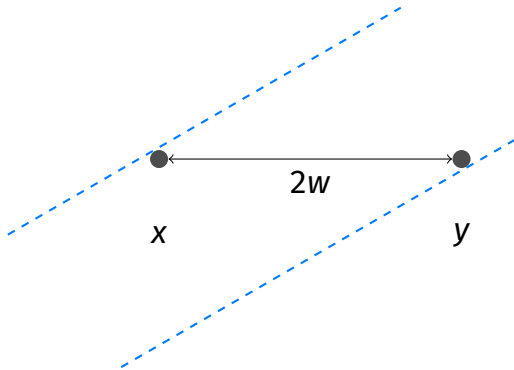
Proof: Far

- Only possible if grid is close to parallel.



Proof: Far

- Angle must be below 30° .



Amplification

- ▶ Lots of points have same hash.
- ▶ To be more selective, randomly select k hash functions for cell id.

$$\text{cell-id}(x) = (h_1(x), h_2(x), \dots, h_k(x))$$

Example: Random Projections

- In case of random projections.

$$\text{cell-id}(\vec{p}) = \left(\underbrace{\left\lfloor \frac{\vec{u}^{(1)} \cdot \vec{p}}{w} \right\rfloor}_{h_1}, \underbrace{\left\lfloor \frac{\vec{u}^{(2)} \cdot \vec{p}}{w} \right\rfloor}_{h_2}, \dots, \underbrace{\left\lfloor \frac{\vec{u}^{(k)} \cdot \vec{p}}{w} \right\rfloor}_{h_k} \right)$$

Collision Probability

- ▶ Remember:

$P(h(x) = h(y)) \geq p_1$ if close.

$P(h(x) = h(y)) \leq p_2$ if far.

- ▶ Collision occurs if $h_i(x) = h_i(y) \forall i \in \{1, \dots, k\}$.
- ▶ Probability of collision...
 - ▶ if close: $\geq p_1^k$
 - ▶ if far: $\leq p_2^k$

Choosing k

- ▶ Want prob. of far points colliding to be small.
- ▶ Say, $1/n$.
- ▶ Set $p_2^k = 1/n$. Then

$$k = \log_{p_2} \frac{1}{n} = -\frac{\log n}{\log p_2}$$

Main Idea

We can use $k = \Theta(\log n)$ hash functions.

Main Idea

When using random projections as hash functions, we can use $k = \Theta(\log n)$ directions. This is usually much less than d .

But wait...

- ▶ Probability of close points colliding is p_1^k .
- ▶ Let $p_1 = p_2^\rho$. We'll have $\rho < 1$, since $p_2 < p_1$.
- ▶ Since $p_2^k = \frac{1}{n}$, we have $p_1^k = \frac{1}{n^\rho}$.
- ▶ This is **very small**.

Banding

- ▶ Before: one set of k hash functions.
- ▶ With **banding**: keep ℓ sets (**bands**) of k hash functions.
- ▶ To query NN of p , find points that are in the same cell as p in *any* of the bands.

Banding

- Probability of at least one match:

$$\underbrace{\frac{1}{n^\rho}}_{\text{collision in band 1}} + \underbrace{\frac{1}{n^\rho}}_{\text{collision in band 2}} + \dots + \underbrace{\frac{1}{n^\rho}}_{\text{collision in band } \ell} = \frac{\ell}{n^\rho}$$

- Want this to be ≈ 1 , so:

$$\ell = n^\rho$$

Main Idea

We should set the number of bands to be n^ρ . ρ depends on c , and is usually not small. For random projections, $\rho \approx .63$.

Analysis

- ▶ How efficient is LSH?
- ▶ Worst case, everything hashes to same bin: $O(n)$.
- ▶ In practice, much better.
- ▶ Requires **a lot** of memory. $\Theta(\ell n)$.

Other Distances

- ▶ LSH works for many different similarity measures.
- ▶ Random projections are for Euclidean distances.
- ▶ But other hashing approaches work for cosine distance, Jaccard distance, etc.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 11

The Johnson-Lindenstrauss Lemma

Why does LSH work?

- ▶ Two approaches to understanding LSH.
- ▶ 1) Hashing view.
- ▶ **2) Dimensionality reduction view.**

Main Idea

The **Johnson-Lindenstrauss Lemma** says that, given n points in $\mathbb{R}^d$, you can reduce the dimensionality to $k \approx \log n$ while still preserving relative distances by randomly projecting onto a set of k unit vectors.

Claim

The **Johnson-Lindenstrauss Lemma** (Informal). Let X be a set of n points in $\mathbb{R}^d$. Let U be a matrix whose $k = O(\log(n)/\epsilon^2)$ rows are Gaussian random vectors in $\mathbb{R}^d$. Then for every $\vec{x}, \vec{y} \in X$,

$$\|\vec{x} - \vec{y}\| \leq (1 \pm \epsilon) \|U\vec{x} - U\vec{y}\|$$

LSH and J-L

- ▶ In LSH, we use $k = O(\log n)$ hash functions.
- ▶ If these hash functions are random projections, the J-L lemma tells that distances are largely preserved.

A Different View of LSH

- ▶ Given $p \in \mathbb{R}^d$, randomly project to $\mathbb{R}^k$ with $k \approx \log n$.
- ▶ Let new coordinates be $(y_1, y_2, \dots, y_k)$.
- ▶ Use standard grid to assign cell id.

Main Idea

LSH (for Euclidean distances) (without banding) can be viewed as dimensionality reduction by random projections, followed by binning into a standard grid.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 5 | Part 12

NN in Practice

In Practice

- ▶ LSH is an important idea.
- ▶ Good performance in practice.
- ▶ But heuristic approaches are often faster.
- ▶ faiss and annoy, among others.

Other Approaches

- ▶ Hierarchical k-means.
- ▶ Product quantization.
- ▶ Navigable small worlds.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 13

Disjoint Sets

Disjoint Sets

- ▶ Often need to keep a collection of **disjoint sets**.
 - ▶ Example: $\{\{4, 6, 2, 0\}, \{1, 3\}, \{5\}\}$
- ▶ May need to union disjoint sets.
- ▶ May need to check if two items are in same set.

Use Case

- ▶ We are given a **stream** of nodes, edges.
- ▶ Want to keep track of CCs at every step.
- ▶ BFS/DFS take $\Theta(V + E)$ time; efficient to compute CCs once, but then need to recompute.

Use Cases

- ▶ Used in Kruskal's algorithm for MST.
- ▶ Used in single linkage clustering.
- ▶ Used in Tarjan's algorithm to find LCA in a tree.

Disjoint Sets, Abstractly

- ▶ A **disjoint sets** ADT represents a collection of disjoint sets.
 - ▶ Example: $\{\{4, 6, 2, 0\}, \{1, 3\}, \{5\}\}$
- ▶ Supports three operations:
 - ▶ `.make_set()`, `.find_set(x)`, `.union(x, y)`
- ▶ Sometimes called a **Union-Find** data type.

Assumption

- ▶ Elements are consecutive integers.
 - ▶ Example: $\{\{4, 6, 2, 0\}, \{1, 3\}, \{5\}\}$
- ▶ Not really a limitation.
 - ▶ Keep dictionary mapping, e.g., string ids to integers.

`.make_set()`

- ▶ Create a new singleton set.
- ▶ Element “id” automatically inferred, returned.

```
>>> ds = DisjointSet()  
>>> ds.make_set()  
0  
>>> ds.make_set()  
1  
>>> ds.make_set()  
2
```


`.union(x, y)`

- ▶ Union sets containing x and y.
- ▶ Updates data structure in-place.

```
>>> ds = DisjointSet()
>>> ds.make_set()
0
>>> ds.make_set()
1
>>> ds.make_set()
2
>>> ds.union(0, 2)
```

.find_set(x)

- ▶ Find **representative** of set containing x.
- ▶ Representative is arbitrary, but same for all items in same set.
- ▶ Used to test if two nodes in same set.
- ▶ Guaranteed to not change unless a union is performed.

```
>>> # ds is {{0}, {1}, {2}}
>>> ds.union(0, 2)
>>> ds.find_set(0)
0
>>> ds.find_set(2)
0
>>> ds.union(0, 1)
>>> ds.find_set(0)
1
>>> ds.find_set(1)
1
>>> ds.find_set(2)
1
```

Today's Lecture

- ▶ How do we implement a disjoint set?
- ▶ We'll introduce the **disjoint set forest** data structure.
- ▶ Talk about two heuristics that make it very efficient.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 14

Disjoint Set Forests

Implementing Disjoint Sets

- ▶ First idea: a **list** of **sets**.

`[{2, 4, 3}, {1, 5}, {0}]`

- ▶ **Problem:** unioning two **sets** takes time linear in size of smaller.

Looking Ahead

- ▶ We'll design data structure so that all operations, including union, take (practically) $\Theta(1)$ time.

The Idea

- ▶ Represent collection as a forest of trees, called a **Disjoint Set Forest**.
- ▶ Example: $\{\{2, 4, 3, 6\}, \{1, 5\}, \{0\}\}$
- ▶ Not unique!

Tree Structure

- ▶ Each node has reference to **parent**.
- ▶ Not a binary tree!

Representing Forests

- ▶ We have several choices:
- ▶ 1) Each node is own **object** with parent attribute.
- ▶ 2) Keep a **list** containing parent of each element.

Approach #1

```
class DSFNode:
```

```
    def __init__(self, parent=None):  
        self.parent = parent
```

- ▶ make_set becomes DSFNode()
- ▶ find_set and union are functions, not methods.
- ▶ They accept DSFNode objects.

Assumption

- ▶ Now, assume that the elements of the disjoint set are **consecutive integers** $0, 1, \dots, n - 1$.
- ▶ We can store the tree in a single list of size n .
- ▶ $\text{arr}[i]$ is parent of element i :

`[2, 0, None, 2, 4]`

Approach #2

```
class DisjointSetForest:
    def __init__(self):
        # self._parent[i] is
        # parent of element i
        self._parent = []

    def make_set(self):
        ...

    def find_set(self, x):
        ...

    def union(self, x, y):
        ...
```

Implementation Notes

- ▶ We'll use the second approach.
- ▶ We can use second representation because elements are consecutive integers.
- ▶ For cache locality, use numpy array, not `list`.

.make_set

```
def make_set(self):  
    # infer new element's "id"  
    x = len(self._parent)  
    self._parent.append(None)  
    return x
```

```
>>> dsf = DisjointSetForest()  
>>> dsf.make_set()  
0  
>>> dsf.make_set()  
1  
>>> dsf.make_set()  
2  
>>> dsf._parent  
[None, None, None]
```

`.find_set(x)`

- ▶ Idea: use the “root” as the representative.

Exercise

Implement `.find_set(x)` recursively.

.find_set

```
def find_set(self, x):  
    if self._parent[x] is None:  
        return x  
    else:  
        return self.find_set(self._parent[x])
```

.union(x, y)

- ▶ Idea: make one root the parent of the other.

`.union(x, y)`

```
def union(self, x, y):  
    x_rep = self.find_set(x)  
    y_rep = self.find_set(y)  
    if x_rep != y_rep:  
        self._parent[y_rep] = x_rep
```

```
>>> # dsf is {{0}, {1}, {2}}  
>>> dsf._parent  
[None, None, None]  
>>> dsf.union(0, 1)  
>>> dsf._parent  
[None, 0, None]  
>>> dsf.union(1, 2)  
>>> dsf._parent  
[None, 0, 0]
```

Analysis

- ▶ `.make_set`: $\Theta(1)$ time³
- ▶ `.union`: depends on `.find_set`
- ▶ `.find_set`: $O(h)$, where h is height of tree

³Amortized, since we're using a dynamic array. But truly $\Theta(1)$ with an over-allocated static array or in the object representation.

Tree Height

- ▶ Trees can be very deep, with $h = O(n)$.
 - ▶ `.find_set` and `.union` can take $\Theta(n)$ time!

- ▶ Example:

```
# dsf is {{0}, {1}, {2}, {3}, {4}}
>>> dsf.union(1, 0)
>>> dsf.union(2, 1)
>>> dsf.union(3, 2)
>>> dsf.union(4, 3)
```

Tree Height

- ▶ But trees can also be shallow, with $h = O(1)$.

- ▶ Example:

```
# dsf is {{0}, {1}, {2}, {3}, {4}}  
>>> dsf.union(0, 1)  
>>> dsf.union(1, 2)  
>>> dsf.union(2, 3)  
>>> dsf.union(3, 4)
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 15

Path Compression and Union-by-Rank

The Bad News

- ▶ We saw that the tree can become very deep.
- ▶ In worst case, `.find_set` and thus `.union` take $\Theta(n)$ time.

Heuristics

- ▶ Now: two heuristics helping trees stay shallow.
- ▶ **Union-by-Rank** and **Path Compression**
- ▶ Together, these result in a **massive** speed up.

Path Compression

- ▶ Idea: if we find a long path during `.find_set`, “compress” it to (possibly) reduce height.

.find_set

```
def find_set(self, x):  
    if self._parent[x] is None:  
        return x  
    else:  
        root = self.find_set(self._parent[x])  
        self._parent[x] = root  
        return root
```

Union-by-Rank

- ▶ Should we `.union(x, y)` or `.union(y, x)`?

Union-by-Rank

- ▶ Placing deeper tree under shallower tree increases height by one.
- ▶ But placing shallower tree under deeper tree doesn't increase height.
- ▶ **Idea:** always place shallower tree under deeper.

Rank

- ▶ We need to keep track of height (**rank**) of each tree.
- ▶ Store rank attribute.
- ▶ $\text{rank}[i]$ is height⁴ of tree rooted at node i .

⁴Exactly the height if path compression isn't used, but upper bound if it is.

Rank

```
class DisjointSetForest:
```

```
    def __init__(self):  
        self._parent = []  
        self._rank = []
```

```
    def make_set(self):  
        # infer new element's "id"  
        x = len(self._parent)  
        self._parent.append(None)  
        self._rank.append(0)  
        return x
```

.union

```
def union(self, x, y):  
    x_rep = self.find_set(x)  
    y_rep = self.find_set(y)  
  
    if x_rep == y_rep:  
        return  
  
    if self._rank[x_rep] > self._rank[y_rep]:  
        self._parent[y_rep] = x_rep  
    else:  
        self._parent[x_rep] = y_rep  
        if self._rank[x_rep] == self._rank[y_rep]:  
            self._rank[y_rep] += 1
```


Note

- ▶ With path compression, rank is no longer *exactly* the height – it is an upper bound.
- ▶ But this is good enough.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 5 | Part 16

Analysis

Analysis of DSF

- ▶ A DSF with path compression and union-by-rank ensures trees are shallow.
- ▶ How does this affect runtime?

Answer

- ▶ Assuming union-by-rank and path compression...
- ▶ In a sequence of m operations, n of which are `.make_sets...`
- ▶ Amortized cost of a single operation is $O(\alpha(n))$.
- ▶ α is the **inverse Ackermann function**, and it is essentially constant.

Inverse Ackermann

$\alpha(n)$	n
0	$n \in [0, 1, 2]$
1	$n = 3$
2	$n \in [4, \dots, 7]$
3	$n \in [8, \dots, 2047]$
4	$n \in [2048, \dots, 2^{2048}]$ and beyond

Proof

- ▶ The formal analysis is quite involved.
- ▶ But we'll provide some intuition.

Union-by-rank Alone

- ▶ Union-by-rank alone ensures height is $O(\log n)$.

```
# dsf is {{0}, {1}, {2}, {3}}  
>>> dsf.union(0, 1)  
>>> dsf.union(2, 3)  
>>> dsf.union(0, 2)
```

Union-by-rank Alone

- ▶ Union-by-rank alone ensures `.find_set` is $O(\log n)$.

Path Compression + U-by-R

- ▶ With path compression, individual `.find_set` calls can take $O(\log n)$.
- ▶ But they massively improve subsequent calls.
 - ▶ For other nodes, too!